

Sesión 4: Árboles de Decisión, kNN y SVM

Tres formas distintas de clasificar

Agustín Gurvich

Introducción a Machine Learning

Qué vimos hasta ahora?

- Regresión lineal, una frontera de decisión **recta** (o un hiperplano)
- Hoy: tres modelos con nuevas visiones sobre el problema

Modelo	Idea
Árbol de decisión	Hacer preguntas de sí o no muy seguidas
kNN	Mirar a los vecinos más cercanos
SVM	Separar las clases con el hiperplano más ancho posible

Parte I

Árboles de decisión

Ejemplo

¿Quién es quién? o **Akinator**: para adivinar un personaje hacemos preguntas de sí o no. Las buenas preguntas descartan a mucha gente, las malas descartan a uno solo.

Ejemplo

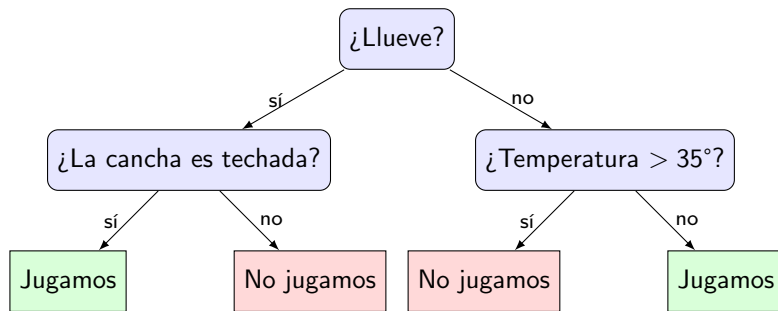
¿Quién es quién? o **Akinator**: para adivinar un personaje hacemos preguntas de sí o no. Las buenas preguntas descartan a mucha gente, las malas descartan a uno solo.

Definición

Un **árbol de decisión** es un modelo que clasifica haciendo una secuencia de preguntas sobre las features. Cada respuesta nos manda por una rama distinta, hasta llegar a una predicción.

Entrenar el árbol es descubrir **qué preguntas hacer y en qué orden**.

¿Salimos a jugar al fútbol?



- **Nodo raíz:** la primera pregunta. **Nodos internos:** preguntas intermedias. **Hojas:** la predicción final.
- **Profundidad:** cuántas preguntas hay, como máximo, desde la raíz hasta una hoja (acá, 2).

Preguntas sobre features numéricas

- Con una feature categórica la pregunta es evidente: “¿llueve?”
- Con una numérica el árbol elige un **umbral**: “¿temperatura ≤ 35 ?”, “¿edad $\leq 17,5$?”
- Cada pregunta corta el espacio con una línea **paralela a un eje**

Que cada grupo quede lo más limpio posible

- Arrancamos con todos los datos en la raíz
- Probamos **todas** las preguntas posibles (cada feature, cada umbral)
- Nos quedamos con la que deja los dos grupos más limpios, cada uno con casi todos los ejemplos de una misma clase
- Repetimos lo mismo dentro de cada grupo, y así hasta terminar

Que cada grupo quede lo más limpio posible

- Arrancamos con todos los datos en la raíz
- Probamos **todas** las preguntas posibles (cada feature, cada umbral)
- Nos quedamos con la que deja los dos grupos más limpios, cada uno con casi todos los ejemplos de una misma clase
- Repetimos lo mismo dentro de cada grupo, y así hasta terminar

Definición

Un nodo es **puro** si todos sus ejemplos son de la misma clase. Para elegir preguntas necesitamos un número que mida qué tan **impuro** es un nodo.

Definición

Si p_k es la proporción de ejemplos de la clase k en un nodo:

$$\text{Gini} = 1 - \sum_k p_k^2$$

Es decir: la probabilidad de equivocarse si etiquetamos un ejemplo al azar siguiendo las proporciones del nodo.

- 10 “sí”, 0 “no”: $\text{Gini} = 1 - (1^2 + 0^2) = 0$ (puro)
- 5 “sí”, 5 “no”: $\text{Gini} = 1 - (0,5^2 + 0,5^2) = 0,5$ (lo más mezclado posible)
- 8 “sí”, 2 “no”: $\text{Gini} = 1 - (0,8^2 + 0,2^2) = 0,32$

Definición

Otra forma de medir el desorden (cortesía de la teoría de la información):

$$H = - \sum_k p_k \log_2 p_k$$

- Nodo puro: $H = 0$
- 50/50 entre dos clases: $H = 1$ (“un bit” de incertidumbre)

Definición

Otra forma de medir el desorden (cortesía de la teoría de la información):

$$H = - \sum_k p_k \log_2 p_k$$

- Nodo puro: $H = 0$
- 50/50 entre dos clases: $H = 1$ (“un bit” de incertidumbre)

Importante

En la práctica Gini y entropía casi siempre eligen las mismas preguntas. scikit-learn usa Gini por defecto (`criterion="gini"`); se puede cambiar con `criterion="entropy"`.

Eligiendo la mejor pregunta: un ejemplo

10 días: 6 jugamos, 4 no. Gini de la raíz = $1 - (0,6^2 + 0,4^2) = 0,48$

Pregunta A: ¿Llueve?

- Sí (4 días): 1 jugamos, 3 no
Gini = 0,375
- No (6 días): 5 jugamos, 1 no
Gini $\approx$ 0,278

Pregunta B: ¿Hace frío?

- Sí (5 días): 3 jugamos, 2 no
Gini = 0,48
- No (5 días): 3 jugamos, 2 no
Gini = 0,48

Eligiendo la mejor pregunta: un ejemplo

10 días: 6 jugamos, 4 no. Gini de la raíz = $1 - (0,6^2 + 0,4^2) = 0,48$

Pregunta A: ¿Llueve?

- Sí (4 días): 1 jugamos, 3 no
Gini = 0,375
- No (6 días): 5 jugamos, 1 no
Gini $\approx$ 0,278

Pregunta B: ¿Hace frío?

- Sí (5 días): 3 jugamos, 2 no
Gini = 0,48
- No (5 días): 3 jugamos, 2 no
Gini = 0,48

Promedio ponderado por cantidad de ejemplos:

- A: $0,4 \cdot 0,375 + 0,6 \cdot 0,278 \approx 0,317$
- B: $0,5 \cdot 0,48 + 0,5 \cdot 0,48 = 0,48$ (¡no mejoró nada!)

Gana “¿Llueve?”: es la pregunta que más reduce la impureza.

¿Cuándo dejamos de preguntar?

Si no lo frenamos, el árbol sigue preguntando hasta que **todas** las hojas son puras.

¿Cuándo dejamos de preguntar?

Si no lo frenamos, el árbol sigue preguntando hasta que **todas** las hojas son puras.

Importante

Un árbol sin límites **memoriza** el set de entrenamiento: llega a 100 % de accuracy en train y le va bastante peor en test. Eso es **overfitting**.

Hiperparámetros para frenarlo:

- `max_depth`: profundidad máxima
- `min_samples_leaf`: mínima cantidad de ejemplos por hoja
- `min_samples_split`: mínima cantidad de ejemplos para partir un nodo

Hiperparámetros vs. parámetros

Definición

Los **parámetros** los aprende el modelo al entrenar (las preguntas y umbrales del árbol, los w de la regresión). Los **hiperparámetros** los elegimos nosotros **antes** de entrenar (`max_depth`, el `learning rate`).

Ejemplo

`max_depth=1`: una sola pregunta, el modelo es demasiado simple (**underfitting**).

`max_depth=None`: memoriza todo (**overfitting**). El punto justo está en el medio, y lo encontramos probando y mirando el resultado en datos que el modelo no vio.

Lo bueno y lo malo de los árboles

Lo bueno

- Se pueden **leer** y explicar: son reglas de sí o no
- No necesitan escalar los datos
- Capturan relaciones no lineales
- Dicen qué features usaron más (*feature importance*)

Lo malo

- Tienden muchísimo al overfitting
- Son **inestables**: cambiar unos pocos datos puede cambiar todo el árbol
- Fronteras en escalones, torpes para cortes diagonales

Lo bueno y lo malo de los árboles

Lo bueno

- Se pueden **leer** y explicar: son reglas de sí o no
- No necesitan escalar los datos
- Capturan relaciones no lineales
- Dicen qué features usaron más (*feature importance*)

Lo malo

- Tienden muchísimo al overfitting
- Son **inestables**: cambiar unos pocos datos puede cambiar todo el árbol
- Fronteras en escalones, torpes para cortes diagonales

Ejemplo

Random Forest: entrenar cientos de árboles distintos y que voten. Corrige gran parte de lo malo, y es de los modelos más usados en la industria para datos en tablas.

Árboles de decisión con scikit-learn

```
from sklearn.tree import DecisionTreeClassifier, plot_tree

modelo = DecisionTreeClassifier(max_depth=3, random_state=0)
modelo.fit(X_train, y_train)

modelo.score(X_test, y_test)      # accuracy
modelo.feature_importances_       # cuanto apporto cada feature

plot_tree(modelo, feature_names=X.columns, filled=True)
```

La interfaz es la misma de siempre: fit, predict, score. Cambia lo que pasa adentro.

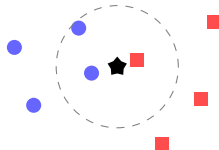
Parte II

kNN y SVM

Definición

kNN (*k-Nearest Neighbors*) clasifica un punto nuevo mirando los k ejemplos de entrenamiento más cercanos y eligiendo la clase que más se repite entre ellos (votación).

- 1 Llega un punto nuevo
- 2 Calcular la distancia a **todos** los puntos de train
- 3 Quedarse con los k más cercanos
- 4 Votar



$k = 3$: 2 azules, 1 roja $\Rightarrow$ azul

La distancia más usada es la **euclídea**:

$$d(a, b) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \cdots + (a_n - b_n)^2}$$

La distancia más usada es la **euclídea**:

$$d(a, b) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots + (a_n - b_n)^2}$$

Importante

Si una feature es “edad” (0 a 100) y otra es “sueldo” (0 a 3.000.000), el sueldo domina por completo la distancia y la edad no cuenta para nada. Con kNN **siempre hay que escalar** los datos antes de entrenar.

¿Qué k elegimos?

- $k = 1$: copia la clase del vecino más cercano. Frontera muy irregular, sensible al ruido $\Rightarrow$ overfitting
- k **muy grande**: siempre predice la clase mayoritaria $\Rightarrow$ underfitting
- En el medio está el punto justo

¿Qué k elegimos?

- $k = 1$: copia la clase del vecino más cercano. Frontera muy irregular, sensible al ruido $\Rightarrow$ overfitting
- k **muy grande**: siempre predice la clase mayoritaria $\Rightarrow$ underfitting
- En el medio está el punto justo

Ejemplo

k es un hiperparámetro, igual que `max_depth`. Se suele usar un k impar para evitar empates cuando hay dos clases.

Lo bueno y lo malo

Lo bueno

- Idea simple y fácil de explicar
- “Entrenar” es rápido: solo guarda los datos
- Fronteras tan complicadas como haga falta

Lo malo

- Predecir es **lento**: compara contra todo el train
- Obligatorio escalar
- Funciona mal con muchísimas features (en dimensiones altas “todos están lejos de todos”)

Definición

A kNN se le llama **lazy learner** porque no aprende nada al entrenar, deja todo el trabajo para el momento de predecir.

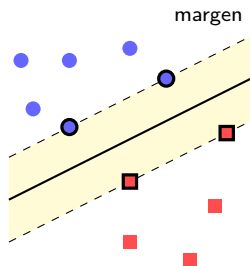
```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_esc = scaler.fit_transform(X_train)
X_test_esc = scaler.transform(X_test)

modelo = KNeighborsClassifier(n_neighbors=5)
modelo.fit(X_train_esc, y_train)
modelo.score(X_test_esc, y_test)
```

Volvemos a la idea de los hiperplanos

- Si las clases se pueden separar con una recta, en general hay **infinitas** rectas que lo hacen
- La regresión logística elige una cualquiera de ellas
- SVM elige la que deja más espacio entre las dos clases



Margen y vectores de soporte

Definición

El **margen** es el ancho de la separación entre las dos clases. SVM busca el hiperplano que **maximiza el margen**.

Definición

Los **vectores de soporte** son los puntos que quedan justo en el borde de la separación (los remarcados en el dibujo). Son los únicos que determinan la frontera: si movemos cualquier otro punto, la frontera no cambia.

Intuición: una separación más ancha deja más espacio de seguridad para datos nuevos que caigan cerca de la frontera.

Margen blando: el hiperparámetro C

En datos reales casi nunca hay una calle perfectamente vacía. Siempre hay algún punto ruidoso del lado equivocado.

- SVM permite que algunos puntos invadan la separación, con una penalización
- C **grande**: casi no tolera errores, separación angosta, riesgo de overfitting
- C **chico**: tolera más errores, separación ancha, modelo más simple

Margen blando: el hiperparámetro C

En datos reales casi nunca hay una calle perfectamente vacía. Siempre hay algún punto ruidoso del lado equivocado.

- SVM permite que algunos puntos invadan la separación, con una penalización
- C **grande**: casi no tolera errores, separación angosta, riesgo de overfitting
- C **chico**: tolera más errores, separación ancha, modelo más simple

Importante

Otra vez lo mismo: un hiperparámetro que maneja cuánto se acomoda el modelo a los datos de entrenamiento.

¿Y si no se pueden separar con una recta?

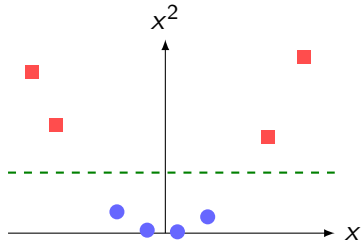


Ningún umbral sobre x
separa azules de rojos

¿Y si no se pueden separar con una recta?



Ningún umbral sobre x
separa azules de rojos



Agregando x^2 como feature,
una recta los separa

Idea: llevar los datos a un espacio con más dimensiones, donde sí se puedan separar con una recta (o un plano).

Definición

Un **kernel** le permite a SVM trabajar *como si* los datos estuvieran en un espacio de más dimensiones, sin tener que calcular esas nuevas features de verdad.

- `kernel="linear"`: frontera recta, como la logística
- `kernel="poly"`: fronteras polinómicas (curvas)
- `kernel="rbf"`: el más usado. Fronteras muy flexibles, que pueden encerrar regiones (círculos, islas)

El truco del kernel

Definición

Un **kernel** le permite a SVM trabajar *como si* los datos estuvieran en un espacio de más dimensiones, sin tener que calcular esas nuevas features de verdad.

- `kernel="linear"`: frontera recta, como la logística
- `kernel="poly"`: fronteras polinómicas (curvas)
- `kernel="rbf"`: el más usado. Fronteras muy flexibles, que pueden encerrar regiones (círculos, islas)

Importante

SVM también usa distancias, así que también hay que escalar los datos!!!

SVM con scikit-learn

```
from sklearn.svm import SVC

modelo = SVC(kernel="rbf", C=1.0)
modelo.fit(X_train_esc, y_train)

modelo.score(X_test_esc, y_test)
modelo.support_vectors_          # los vectores de soporte
```

SVM fue el modelo estrella antes de que llegaran las redes neuronales profundas, y sigue funcionando muy bien con datasets medianos.

¿Cuál uso?

	Logística	Árbol	kNN	SVM
Frontera	recta	escalones	libre	recta o curva
¿Hay que escalar?	conviene	no	sí	sí
¿Se puede interpretar?	bastante	mucho	poco	poco
Velocidad al predecir	rápida	rápida	lenta	media
Hiperparámetro clave	C	<code>max_depth</code>	k	C , kernel

¿Cuál uso?

	Logística	Árbol	kNN	SVM
Frontera	recta	escalones	libre	recta o curva
¿Hay que escalar?	conviene	no	sí	sí
¿Se puede interpretar?	bastante	mucho	poco	poco
Velocidad al predecir	rápida	rápida	lenta	media
Hiperparámetro clave	C	<code>max_depth</code>	k	C , kernel

Importante

No hay un modelo que gane siempre. La única forma de saber cuál anda mejor en un problema es probar varios y compararlos en datos que no vieron.

- Árbol de decisión: preguntas de sí o no elegidas para que cada grupo quede lo más puro posible (Gini o entropía)
- kNN: clasificar por votación de los k vecinos más cercanos
- SVM: la frontera con el margen más ancho; con kernels, también fronteras curvas
- Los tres tienen hiperparámetros que regulan el equilibrio entre underfitting y overfitting

¿Preguntas?